

DOMAIN: *WEB3*
TEAM: *WEB3 74*
TEAM NAME: *oneANDzero*

PROJECT:
BLOCK-DOCK

MENTORS	<i>Kashyap</i>	<i>Mohith</i>
Team members	SRN	
Ashwin S	R22EJ078	
Adarsh BM	R22EF005	
Sanjana SN	R22EJ052	
Vishal Sagar Murthy	R22EQ044	

Problem Statement:

Urban municipalities often face challenges related to grievance management, fund allocation, and tax collection. The traditional systems lack transparency, and there are frequent delays in resolving issues due to bureaucratic inefficiencies. Urban residents are also unaware of how taxes are utilized, and there is limited visibility into ongoing projects and funds received from the government.

Solution:

Block-Dock

A Blockchain-Based Urban Grievance Redressal and Fund Management System

The **Block-Dock** project aims to develop a blockchain-based governance platform that enhances transparency, efficiency, and security in administrative processes. The platform involves three types of users: **Users**, **AdminHead**, and **AdminGovt**, each with distinct roles and responsibilities. By leveraging blockchain technology, the system ensures immutable records, decentralized control, and real-time visibility of all interactions.

Approach

Smart Contract Development:

1. **Role Definition:** The roles of AdminGovt, AdminHead, and Users were implemented using a role-based access control system to ensure data integrity and restricted operations.
 2. **Token System:** An ERC20-compatible token was developed for fund management. AdminGovt can transfer tokens to AdminHead upon request, with transaction history logged on-chain.
 3. **Grievance Management:** Grievances were modeled as structs, including fields for user details, description, and status. Grievances are submitted with an initial status of **Pending** and updated to **Accepted** or **Rejected** by AdminHead.
 4. **Project Management:** Projects were stored on-chain with details such as name, due_time, and contractor_name. Projects could only be created by AdminHead and were visible to all Users.
-

Frontend Development:

The user interface was built using **React** with seamless MetaMask integration for wallet-based authentication. Features were role-specific:

- **Users:**
 - Register and log in using their wallet.
 - File grievances and view their status.
 - View project details and AdminHead information.
 - **AdminHead:**
 - Request tokens from AdminGovt with the purpose and amount.
 - Manage grievances by updating their status.
 - Create and view projects.
 - Search user information dynamically.
 - **AdminGovt:**
 - Approve or deny token requests.
 - Monitor grievance records and user details.
 - View all projects and transaction histories.
-

Blockchain Integration

1. **Wallet Authentication:** MetaMask was integrated for wallet connection, enabling role-specific functionalities.
 2. **Smart Contract Deployment:** The contract was deployed on a testnet to ensure low-cost testing and debugging. Functions were tested to ensure security and seamless execution.
-

Tech Stack

- **Solidity:** Used for developing the core smart contracts managing roles, grievances, projects, and token transactions.
 - **Sepolia Testnet:** Utilized as the blockchain environment for deploying and testing the smart contract.
 - **MetaMask:** Used for user authentication and interaction with the smart contract, enabling role-based operations like grievance filing and token transfers.
 - **Hardhat:** Framework for compiling, testing, and deploying the Solidity contracts efficiently. Includes functionality for debugging and local blockchain simulation.
 - **React.js:** Framework for building the user interface, ensuring a dynamic and responsive experience. MetaMask integration enabled users to connect wallets and access role-specific features.
 - **CSS:** Used for styling the frontend to ensure an intuitive and visually appealing user interface.
 - **Node.js:** Used for setting up the server-side environment and managing the interaction between the frontend and the blockchain.
-

Prerequisites

Basics of blockchain concepts (e.g., smart contracts, wallets, tokens).
Understanding of React.js and JavaScript.

Software/Tool Installations:

- Node.js and npm installed for frontend and backend setup.
- MetaMask browser extension for wallet integration.
- Hardhat for smart contract development and deployment.

Software and Tool Versions

Solidity v0.8.26

React.js v18.2.0

Node.js v18.x

Hardhat v2.17.0

MetaMask

Sepolia testnet

Code Snippets

EM.sol: (smart contract)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.26;

contract GrievanceSystem {
    address public adminGovt;
    address public adminHead;
    mapping(address => uint256) public tokenBalance;
    uint256 public totalSupply = 10000; // AdminGovt starts with 10,000 tokens
    struct User {
        string name;
```

```

    string email;
    string dob;
    string mobile;
}

mapping(address => User) public users;
struct Grievance {
    string name;
    string details;
    string status; // PENDING, ACCEPTED, REJECTED
    address user;
}
Grievance[] public grievances;
struct Project {
    string pname;
    string details;
    uint256 customFund;
    string status; // ONGOING, DONE
    address adminHead;
}
Project[] public projects;
struct FundRequest {
    uint256 amount;
    string status; // NOTPAID, PAID
    address adminHead;
}
FundRequest[] public fundRequests;
event TokensTransferred(address from, address to, uint256 amount);
event FundRequestCreated(address adminHead, uint256 amount);
event FundRequestStatusChanged(uint256 requestId, string status);
event ProjectCreated(address adminHead, string pname, uint256 customFund);
event ProjectStatusChanged(uint256 projectId, string status);
constructor() {
    adminGovt = msg.sender; // Set the deployer as AdminGovt
    tokenBalance[adminGovt] = totalSupply; // Assign all tokens to AdminGovt
}
modifier onlyAdminGovt() {
    require(msg.sender == adminGovt, "Only AdminGovt can perform this action.");
}

```

```

    _;
}

modifier onlyAdminHead() {
    require(msg.sender == adminHead, "Only AdminHead can perform this action.");
    _;
}

address[] public registeredUsers; // Array of registered users' addresses

function registerUser(address _userAddress, string memory _name, string memory _email, string memory
_dob, string memory _mobile) public {
    users[_userAddress] = User({
        name: _name,
        email: _email,
        dob: _dob,
        mobile: _mobile
    });
    registeredUsers.push(_userAddress); // Add to registered users array
}

function fileGrievance(string memory _name, string memory _details) public {
    grievances.push(Grievance({
        name: _name,
        details: _details,
        status: "PENDING",
        user: msg.sender
    }));
}

function viewGrievances() public view returns (Grievance[] memory) {
    return grievances;
}

function viewAdminHeadDetails() public view returns (address) {
    return adminHead;
}

function viewBalanceAdminHead() public view onlyAdminHead returns (uint256) {
    return tokenBalance[msg.sender];
}

function viewAndEditGrievances(uint256 grievanceIndex, string memory _status) public onlyAdminHead {
    Grievance storage grievance = grievances[grievanceIndex];
    require(grievance.user != address(0), "Invalid grievance index.");
    grievance.status = _status;
}

function requestFunds(uint256 _amount) public onlyAdminHead {

```

```

    require(_amount > 0, "Amount must be greater than 0.");
    fundRequests.push(FundRequest({
        amount: _amount,
        status: "NOTPAID",
        adminHead: msg.sender
    }));
    emit FundRequestCreated(msg.sender, _amount);
}

function createProject(string memory _pname, string memory _details, uint256 _customFund) public
onlyAdminHead {
    require(tokenBalance[adminGovt] >= _customFund, "Insufficient balance in AdminGovt to allocate custom
funds.");

    tokenBalance[adminGovt] -= _customFund;
    tokenBalance[adminHead] -= _customFund;

    projects.push(Project({
        pname: _pname,
        details: _details,
        customFund: _customFund,
        status: "ONGOING",
        adminHead: msg.sender
    }));

    emit ProjectCreated(msg.sender, _pname, _customFund);
    emit TokensTransferred(adminGovt, adminHead, _customFund); // Emit event for the transfer
}

function editProjectStatus(uint256 projectId, string memory _status) public onlyAdminHead {
    Project storage project = projects[projectId];
    require(project.adminHead == msg.sender, "Only the creator can edit this project.");
    project.status = _status;
    emit ProjectStatusChanged(projectId, _status);
}

function viewAllUsers() public view returns (User[] memory) {
    User[] memory allUsers = new User[](registeredUsers.length);
    for (uint i = 0; i < registeredUsers.length; i++) {
        allUsers[i] = users[registeredUsers[i]];
    }
    return allUsers;
}

```

```

function viewBalanceAdminGovt() public view onlyAdminGovt returns (uint256) {
    return tokenBalance[msg.sender];
}

function assignAdminHead(address _adminHead) public onlyAdminGovt {
    adminHead = _adminHead;
}

function viewFundRequests() public view returns (FundRequest[] memory) {
    return fundRequests;
}

function sendTokensToAdminHead(uint256 _requestId) public onlyAdminGovt {
    require(_requestId < fundRequests.length, "Invalid request ID.");
    FundRequest storage request = fundRequests[_requestId];
    require(keccak256(abi.encodePacked(request.status)) == keccak256(abi.encodePacked("NOTPAID")),
"Request already paid.");
    require(tokenBalance[adminGovt] >= request.amount, "Insufficient balance to send tokens.");
    tokenBalance[adminGovt] -= request.amount;
    tokenBalance[request.adminHead] += request.amount;
    request.status = "PAID";
    emit FundRequestStatusChanged(_requestId, "PAID");
    emit TokensTransferred(adminGovt, request.adminHead, request.amount);
}

// View all projects (AdminGovt)
function viewAllProjects() public view returns (Project[] memory) {
    return projects;
}
}

```

Output/UI Snapshots

BLOCK-DOCK

Block-Dock is a blockchain-powered platform designed to bring transparency, efficiency, and accountability to urban governance. We specialize in grievance redressal, tax collection, and fund management systems, ensuring seamless interaction between citizens and municipalities.

Our platform empowers citizens to file grievances, track municipal taxes, and monitor urban development projects. For municipalities, we provide tools for efficient fund allocation and project management, fostering trust and collaboration with the community.

Services We Provide

File Grievance

Easily file and track the status of your grievances.

Government Funds

View Govt funds sent and received by AdminGovt and AdminHead.

Fund Management

Track government funds allocated for urban projects.

Ongoing Projects

Monitor progress on ongoing municipal projects.

Block-Dock

Welcome to E-Municipality,Vishwa!

File Grievance

Submit your complaints or issues for the municipality's attention.

[File Grievance](#)

Ongoing Projects

Track the progress and updates of current municipal projects.

[Ongoing Projects](#)

Government Funds

View details of funds received from the government.

[Government Funds](#)

Admin Details

Reach out to us for any queries or assistance.

[Contact Us](#)

My Grievances

View and manage grievances you have submitted.

[My Grievances](#)

All Grievances

View all grievances made by all users.

[All Grievances](#)

Block-Dock

File Your Grievance

Please fill out the form below to submit your grievance regarding city services.

Name

Enter your name

Details

Enter the details

Submit

Block-Dock

Edit Grievances

Name	Details	Status	Actions
Rabi	Destroy Clg	REJECTED	<button>Accept</button> <button>Reject</button>
Adarsh	Sth	ACCEPTED	<button>Accept</button> <button>Reject</button>
Ashwin Sudhakar	Some issue	PENDING	<button>Accept</button> <button>Reject</button>
Ashwin	some more issue	PENDING	<button>Accept</button> <button>Reject</button>
N/A	sth sth	PENDING	<button>Accept</button> <button>Reject</button>
N/A	Hello world	PENDING	<button>Accept</button> <button>Reject</button>
N/A	Simple problem	PENDING	<button>Accept</button> <button>Reject</button>
Rabi rahul	Sth	PENDING	<button>Accept</button> <button>Reject</button>
Vishwa B	ABC	PENDING	<button>Accept</button> <button>Reject</button>

Block-Dock

Balance: 115

Request Funds

Amount (in Tokens)

Request Fund

Received Fund Requests

Request Amount (ETH)	Status
100	PAID
10	NOTPAID
10	NOTPAID
10	NOTPAID
100	PAID

Block-Dock

Balance: 9715 RS

Received Fund Requests

Amount (in Tokens)	Status	Pay
100	PAID	<button>Pay</button>
10	NOTPAID	<button>Pay</button>
10	NOTPAID	<button>Pay</button>
10	NOTPAID	<button>Pay</button>
100	PAID	<button>Pay</button>

Block-Dock

Balance: 115

Fund Allocation

Add New Project

Create Project

Ongoing Projects

Project Name	Details	Amount (in Tokens)	Status	Actions
Project-1	Road Construction near reva University	10	DONE	
Project-2	Something	10	DONE	
sth	sth	50	ONGOING	Mark as Done
Project-3	ABC	15	ONGOING	Mark as Done

Block-Dock

Municipality Administration Details

Municipality Head (Admin-Head)

Role: Oversees funds, allocates funds for projects, assigns contractors, and views individual tax details.

Authentication: MetaMask authentication for secure access.

Key Actions: Manages fund allocation and project licensing to ensure transparent fund utilization.

Wallet: 0xE1d7C37f7fa7e189e0191c02379fE97BcB1c5984

Government Department Officer

Role: Manages fund transfers to the municipality, monitors tax collection and fund allocation, and oversees financial integrity.

Authentication: MetaMask authentication for secure access.

Key Actions: Ensures that all allocated funds are transparent and accessible for public viewing.

Wallet: 0x0e6042dc08B0a6A0747b95dab1C12abD76d3cfC5

Individual Contribution

<i>Ashwin</i>	Designed smart contract. Worked on back-end logic to connect the blockchain with the front end.(for all pages)
<i>Vishal</i>	Assisted in back-end development. Deployed the smart contract on the Sepolia testnet. And assisted in interaction with contract.(for all pages)
<i>Adarsh</i>	Designed and developed complete front-end user pages(10-11) on react.
<i>Sanjana</i>	Designed and developed complete front-end AdminGovt, AdminHead pages(10) on react.

Quick Access links

Github: [Block-Dock](#)
Google-Drive: [Block-Dock](#)

Reference links

- Hardhat documentation: <https://hardhat.org/hardhat-runner/docs/getting-started>
- Web3js documentation: <https://web3js.readthedocs.io/en/v1.10.0/>
- Ethers.js documentation: <https://docs.ethers.org/v5/getting-started/>
- OpenAI: <https://chat.openai.com/>
- Youtube: <https://www.youtube.com/>
- Metamask: <https://developer.metamask.io/>